

AI Operations — Module 1: Experiment Management & Reproducibility

Rohan Sirsewad · github.com/Ronny33/aiops-module1

1. Technical Debt Diagnosis

(a) **Rounding change broke an unrelated feature → Entanglement (CACE).** Feature weights are optimised jointly against one loss, so coarsening delivery-time values shifts the loss surface and the optimiser re-balances every other feature. Nothing in the favorite-restaurants code changed; the damage travelled through the shared objective.

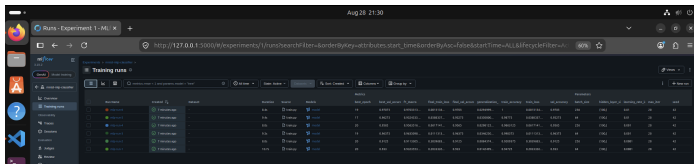
(b) **Dashboard team reading the raw output table → Undeclared Consumers.** An invisible dependency nobody documented. The ML team can no longer change output format or model version safely, and only finds out when something downstream breaks.

(c) **14 undocumented shell scripts → Configuration & Glue-Code Debt.** Hyperparameters buried in individual scripts with no single source of truth, execution order only in someone's head, intermediate outputs unversioned — the “pipeline jungle” from the lecture.

Mitigation for (c): replace the scripts with a declarative `dvc.yaml`, the orchestration tool the setup is missing. Each stage declares `cmd`, `deps`, `params`, `outs`, so the DAG becomes a reviewable file in Git; scattered hyperparameters move into one `params.yaml`, killing the configuration half of the debt; and `dvc repro` re-runs only stages whose inputs changed. Wrapping it as an MLflow Project (MLproject) adds documented entry points so a new engineer runs `mlflow run` without reading any internals.

2. MLflow Experiment Comparison

MLPClassifier on MNIST-784, sweeping `learning_rate_init` ∈ {10⁻⁴, 10⁻³, 10⁻²} × `batch_size` ∈ {64, 256}; one hidden layer of 100, 20 epochs, seed 42, 16k/4k split.



Best run was **mlp-run-6** (lr=0.01, batch=256), `val_accuracy`=0.9708 at epoch 19. lr=0.01 gets most of the way there within three epochs and the larger batch smooths the gradient noise a big learning rate would otherwise cause — visible in `mlp-run-5`, same lr but batch 64, where `train_loss` bounces (0.0874, 0.0896, 0.0702, 0.0780) instead of falling cleanly. Clearest overfitting is **mlp-run-3** (lr=0.001, batch=64): between epochs 13 and 20 `train_loss` drops 0.0361→0.0111, a 69% reduction, while `val_accuracy` barely moves, 0.9635→0.9637. The model fits the training set harder and gets nothing back on validation — it has stopped generalizing and started memorizing. `mlp-run-6` is a milder case, ending at `train_loss`=0.0015.

`learning_rate_init` has the bigger effect: averaging `best_val_accuracy` per value gives 0.9228→0.9601→0.9668, a 4.40 point spread, against 0.9531→0.9466 for batch size, only 0.65 points — roughly 7× more. Batch size mainly changed convergence smoothness and wall-clock time; learning rate decided whether a run converged at all within the 20-epoch budget. They interact though: at low lr the smaller batch wins, at lr=0.01 the bigger one does.

Logging code (full script: `train.py`). Params logged once; metrics logged *per epoch* via `step=`, which is what produces the trend the analysis relies on:

```
mlflow.log_param("learning_rate_init", lr)
mlflow.log_param("batch_size", bs)
mlflow.log_param("seed", SEED)
for epoch in range(1, max_iter+1):
    model.partial_fit(X_tr, y_tr, classes=C)
```

```
mlflow.log_metric("train_loss", loss, step=epoch)
mlflow.log_metric("val_accuracy", acc, step=epoch)
mlflow.log_metric("best_val_accuracy", max(accs))
```

3. DVC Versioning & Rollback

`build_index.py` writes a sorted, deterministic CSV of filenames from the Lecture 2 cats-vs-dogs set. v1 = 1800 rows (1801 lines); after `new-labels.zip`, v2 = 2800 rows (2801 lines). *The brief's "2801" is the line count, matching its own description of v1 as "1800 rows + a header line".* Remote is SSH; v1/v2 are Git-tagged.

	lines	md5
v2 (before)	2801	3cc34b3e...bbeb
v1 (after rollback)	1801	3367084a...d51a

`git checkout v1.0` then `dvc checkout` returns 2801→1801 lines, and the md5 matches the hash v1 was pushed with exactly. Row count proves length; the hash proves identity. Two commands are needed because `git checkout` only swaps the small `.dvc` pointer holding an md5 — the CSV was never in Git — and `dvc checkout` restores the matching blob from the cache. Committing v2 produced a *two-line* Git diff for 1000 new images, which is the separation DVC exists for. Full transcript in the repo README.

4. Reproducibility Drill (Partner B)

Partner A shared only a repo link. Running the prescribed protocol, three of five steps failed: `dvc checkout` (empty cache on fresh clone, expected), `dvc pull` ([Errno 101] Network is unreachable — the remote pointed at a VirtualBox NAT address, which is outbound-only and unroutable from any other machine; confirmed via both ping and a direct port-22 check), and `mamba env create` (no `environment.yml` exists; the repo ships MLflow-Projects `python_env.yaml`, which mamba cannot parse). Resolved after Partner A switched her VM to bridged networking, plus a local `dvc remote modify --local ssh-remote user vboxuser` override. The pulled dataset verified byte-identical (md5 7396020f...fa8d0, 109,640,201 bytes).

	Partner A	Partner B
accuracy	≈0.81	0.809000
f1_macro	≈0.80	0.804191

Verdict: MATCHED, tolerance 0.01 absolute (A reported to 2 d.p., so no tighter claim is supportable). Logged as a `reproduction_note` tag on the rerun.

What the drill exposed: data that was never actually shareable; no `environment.yml`; and no `git-commit` tag, so no link from run to code revision. Separately, `--seed` has no default in `train.py`, so omitting it makes `train_test_split` non-deterministic — masked by MLproject supplying `seed: 42`, and easy to miss because the model's own `random.state` is hardcoded.